4

Instruções de controle: Parte 1

4.1 Introdução

4.2 Algoritmos

4.3 Pseudocódigo

4.4 Estruturas de controle

4.5 A instrução de única seleção `if`

4.6 A instrução de seleção dupla `if…else`

4.7 Instrução de repetição `while`

4.8 Formulando algoritmos: repetição controlada por contador

4.9 Formulando algoritmos: repetição controlada por sentinela

4.10 Formulando algoritmos: instruções de controle aninhadas

4.11 Operadores de atribuição compostos

4.12 Operadores de incremento e decremento

4.13 Tipos primitivos

4.14 Conclusão

OBJETIVOS

Neste capítulo você aprenderá:

- Como utilizar técnicas básicas de solução de problemas.
- Como desenvolver algoritmos por meio do processo de refinamento passo a passo de cima para baixo.
- Como usar instruções de seleção `if` e `if… else` para escolher entre ações alternativas.
- Como utilizar a instrução de repetição `while` para executar instruções em um programa repetidamente.
- Como utilizar repetição controlada por contador e repetição controlada por sentinela.
- Como utilizar operadores de atribuição, incremento e decremento.

4.2 Algoritmos

- **Algoritmos**
  - **Ações a executar**
  - **Ordem em que essas ações executam**
- **Controle de programa**
  - **Especifica a ordem em que as instruções (ações) executam em um programa**

4.3 Pseudocódigo

- **Pseudocódigo**
  - **Uma linguagem informal semelhante ao nosso idioma**
  - **Ajuda programadores a desenvolver algoritmos**
  - **Não executa em computadores**
  - **Podem conter entrada, saída e cálculo**
  - **Geralmente sem declarações de variáveis**

4.4 Estruturas de controle

- **Execução sequencial**
  - **Instruções em um programa são executadas uma após a outra na ordem em que são escritas**
- **Transferência de controle**
  - **Especificar qual a próxima instrução a executar não é necessariamente a próxima na sequência**
  - **Pode ser permitida pela instrução `goto`**
    - **Programação estruturada elimina instruções `goto`**

4.4 Estruturas de controle (Cont.)

- **Pesquisa de Bohm e Jacopini**
  - **Provaram que instruções `goto` são desnecessárias**
  - **Provaram que todos os programas poderiam ser escritos com apenas 3 estruturas de controle**
    - **Estrutura de sequência,**
    - **Estrutura de seleção e**
    - **Estrutura de repetição.**

4.4 Estruturas de controle (Cont.)

- **Diagrama de atividades UML (www.uml.org)**
  - **Modela o fluxo de trabalho (ou atividade) de uma parte de um sistema de software**
  - **Símbolos do estado da ação (retângulos com os lados direito e esquerdo substituídos por arcos curvados para fora)**
    - **Representam ações a realizar**
  - **Diamantes**
    - **Símbolos de decisão (seção 4.5)**
    - **Símbolos de agregação (seção 4.7)**

4.4 Estruturas de controle (Cont.)

- **Pequenos círculos**
  - **Círculo sólido representa o estado inicial da atividade**
  - **Círculo sólido cercado por um círculo vazio representa o estado final da atividade**
- **Setas de transição**
  - **Indicam a ordem em que as ações devem ocorrer**
- **Notas (retângulos com cantos superiores direitos dobrados)**
  - **Explicam o propósito dos símbolos (como comentários em Java)**
  - **São conectados pelos símbolos que descrevem através de linhas pontilhadas**

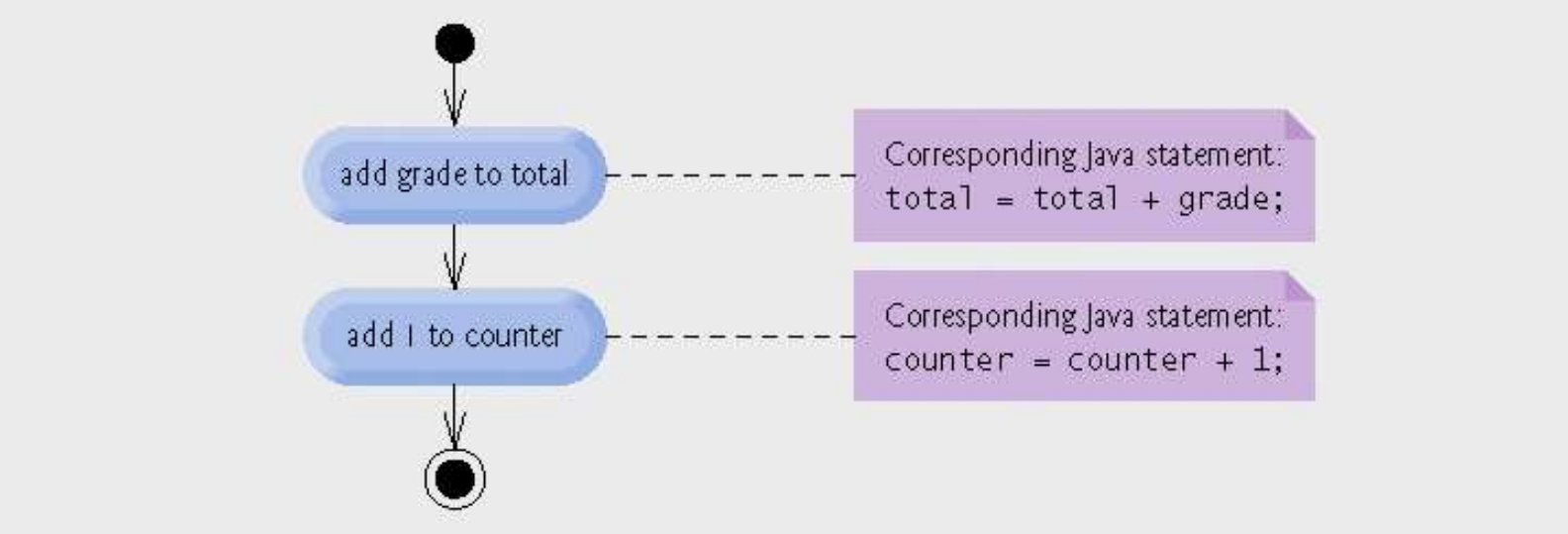


Fig. 4.1 | Diagrama de atividades da estrutura de sequência.

4.4 Estrutura de controle (Cont.)

- **Instruções de seleção**
  - **Instrução `if`**
    - **Instrução de uma única seleção**
  - **Instrução `if`...`else`**
    - **Instrução de seleção dupla**
  - **Instrução `switch`**
    - **Instrução de seleção múltipla**

4.4 Estruturas de controle (Cont.)

- **Instruções de repetição**
  - **Também conhecidas como instruções de loop**
  - **Executa uma ação repetidamente contanto que uma condição seja verdadeira**
  - **Instrução `while`**
    - **Realiza a ação no seu corpo zero ou mais vezes**
  - **Instrução `do`...`while`**
    - **Realiza a ação no seu corpo uma ou mais vezes**
  - **Instrução `for`**
    - **Realiza a ação no seu corpo zero ou mais vezes**

4.4 Estruturas de controle (Cont.)

- **Java possui três tipos de estruturas de controle**
  - **Instrução de sequência,**
  - **Instruções de seleção (três tipos) e**
  - **Instruções de repetição (três tipos)**
  - **Cada programa é composto dessas estruturas de controle**
    - **Empilhamento de instruções de controle**
      - **Instruções de controle de entrada única/saída única**
    - **Aninhamento de instruções de controle**

4.5 Instrução de única seleção `if`

- **Instruções `if`**
  - **Executa uma ação se a condição especificada for `true`**
  - **Pode ser representada por um símbolo de decisão num diagrama de atividade UML**
    - **Setas de transição que saem de um símbolo de decisão têm uma condição de guarda**
      - **O fluxo de trabalho segue a seta de transição a qual a condição de guarda é true**

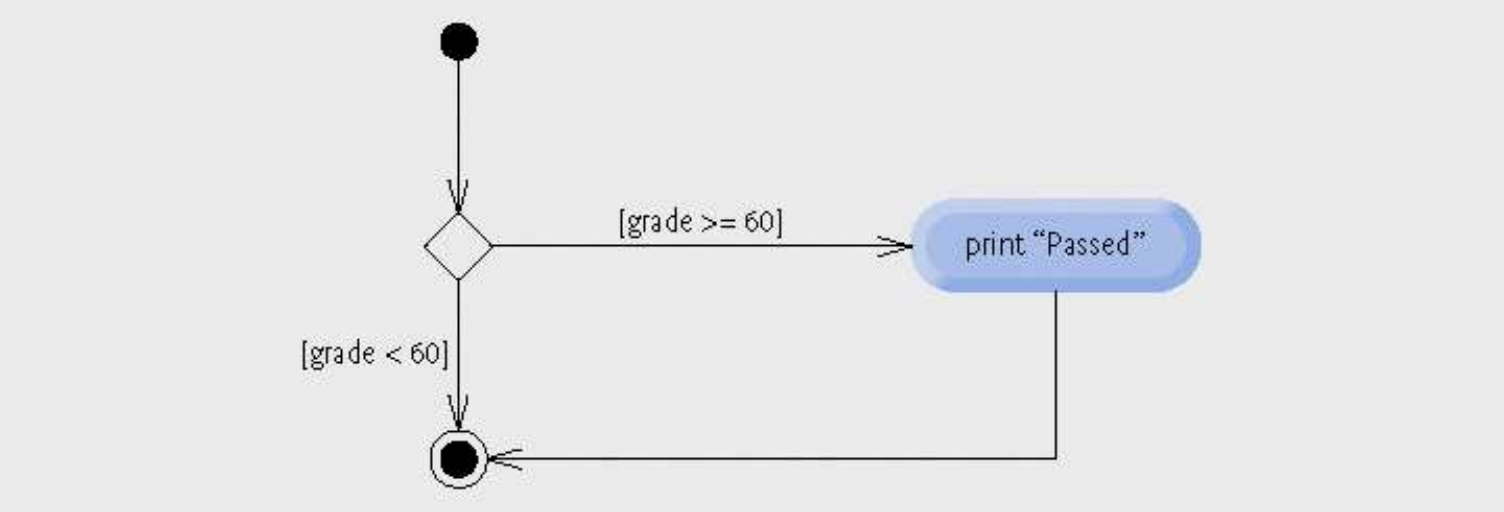
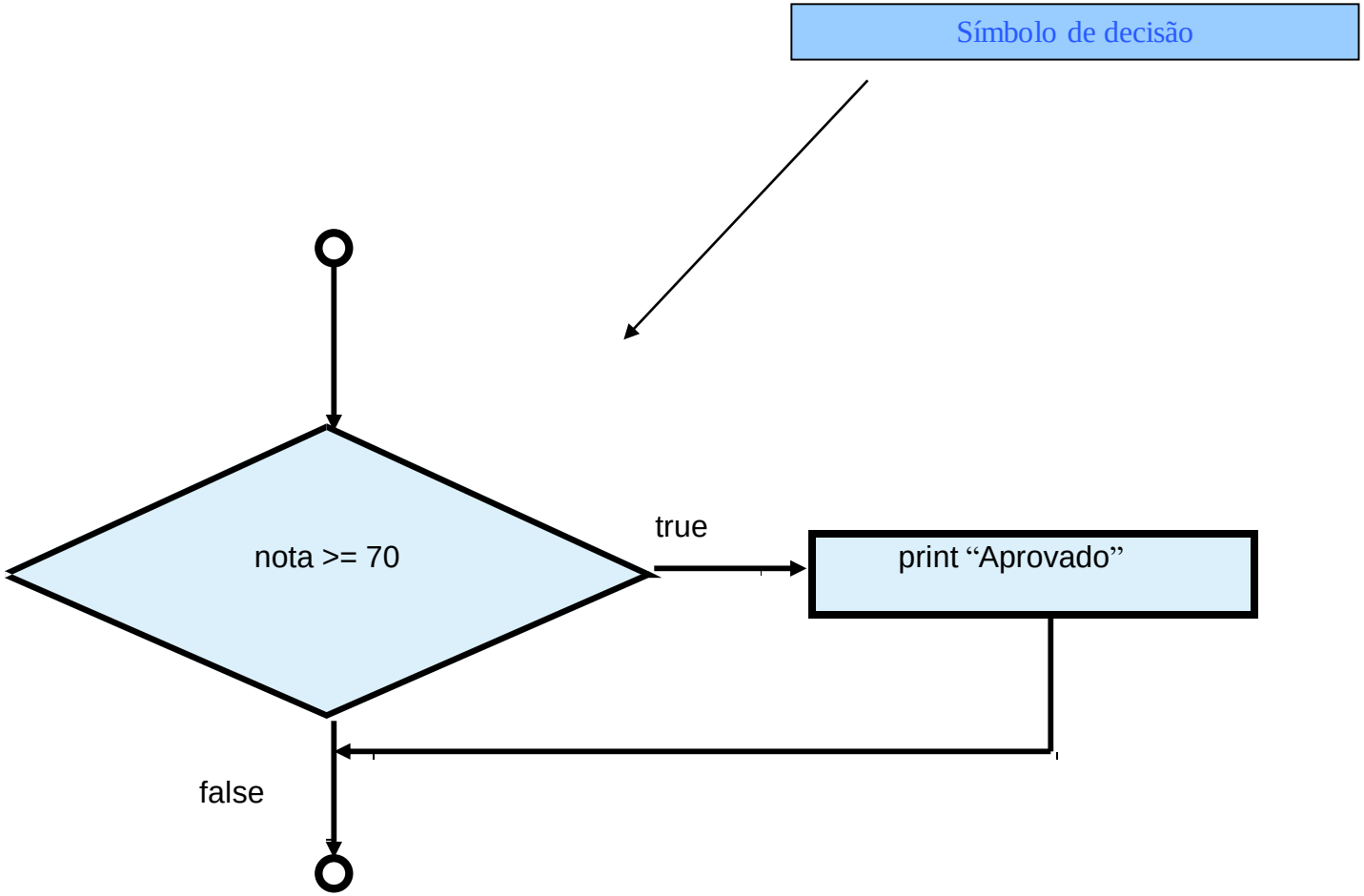


Fig. 4.2 | Diagrama de atividades UML de uma instrução de seleção única `if`.



Fluxograma de uma instrução de seleção única `if`.

4.6 Instrução de seleção dupla `if`…`else`

- **Instrução `if`…`else`**
  - **Executa uma ção se a condição especificada for `true` ou diferente caso a condição seja `false`**
- **Operador condicional (`? :`)**
  - **Único operador ternário (três operandos) em Java**
  - **`? :` e seus três operandos formam uma expressão condicional**
    - **A expressão condicional avalia o segundo operando caso o primeiro operando seja `true`**
    - **A expressão condicional avalia o terceiro operando caso o primeiro operando seja `false`**

Boa prática de programação 4.1

Recue as duas instruções do corpo de uma instrução `if - else`.

Boa prática de programação 4.2

Se existirem vários níveis de recuo, cada nível deve ser recuado pela mesma quantidade adicional de espaços.

Boa prática de programação 4.3

As expressões condicionais são mais difíceis de ler do que as instruções `if-else` e devem ser utilizadas para substituir somente instruções `if-else` simples que escolhem entre dois valores.

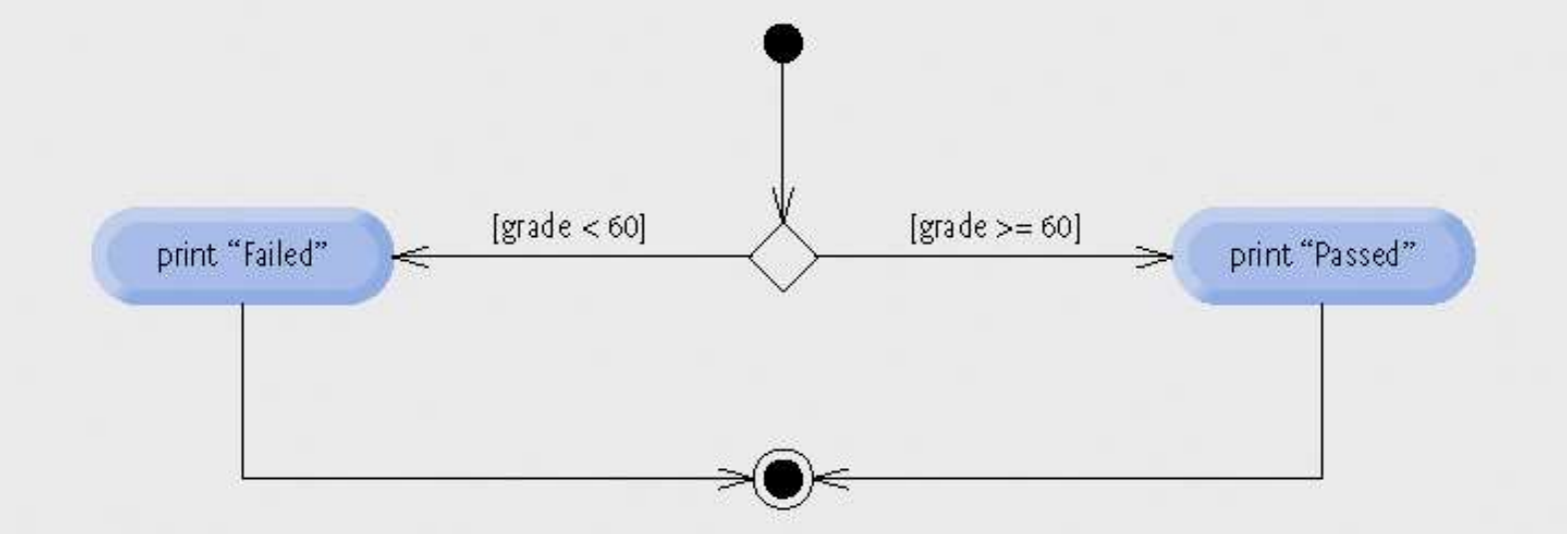
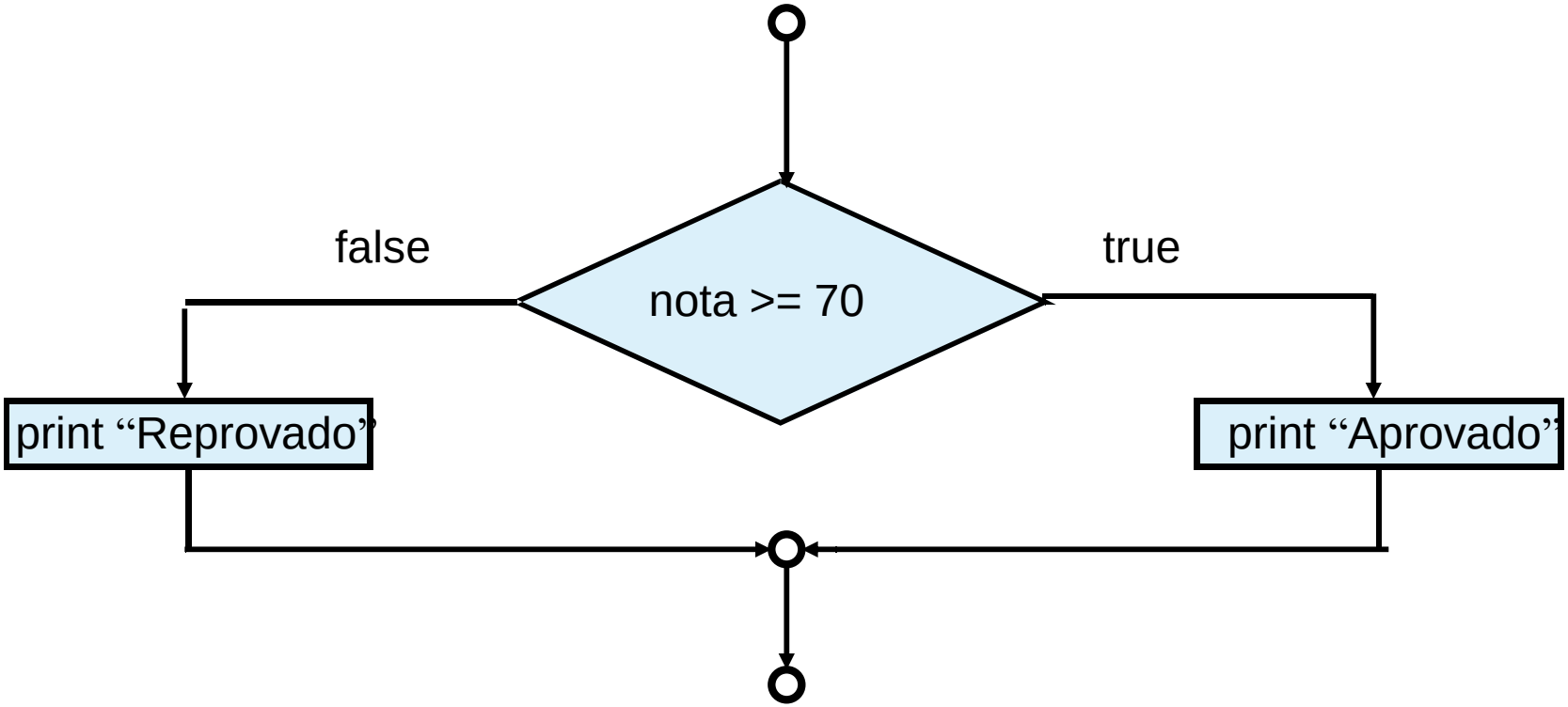


Fig. 4.3 | Diagrama de atividades UML de instrução de seleção dupla `if`-else.



Fluxograma de uma instrução de seleção dupla if/else

4.6 Instrução de seleção dupla `if`...`else` (Cont.)

- **Instruções `if`...`else` aninhadas**
  - **Instruções `if`...`else` podem ser inseridas em outras instruções `if`...`else`**
- **Problema do `else` oscilante**
  - **`else`s são associados com o `if` imediatamente anterior a menos que sejam especificadas chaves `{ }`**
- **Blocos**
  - **Chaves `{ }` associam instruções em blocos**
  - **Blocos podem alterar instruções individuais como um corpo de um `if`**

4.6 Instrução de seleção dupla `if`…`else` (Cont.)

- **Erros de lógica**
  - **Erros fatais de lógica fazem com que um programa falhe e finalize prematuramente**
  - **Erros de lógica não-fatais causa resultados incorretos**
- **Instruções vazias**
  - **Representada por um ponto-e-vírgula (`;`) colocado onde normalmente uma instrução estaria**
  - **Podem ser usadas num corpo de um `if`**

Erro comum de programação 4.1

Esquecer uma ou ambas as chaves que delimitam um bloco pode levar a erros de sintaxe ou erros de lógica em um programa.

Boa prática de programação 4.4

Sempre utilizar as chaves em uma instrução `if...else` (ou outra) ajuda a evitar uma omissão acidental, especialmente ao adicionar instruções à parte `if` ou à parte `else` mais tarde. Para evitar omitir uma ou as duas chaves, alguns programadores digitam as chaves de abertura ou fechamento de blocos antes de digitar as instruções individuais dentro das chaves.

Erro comum de programação 4.2

Colocar um ponto-e-vírgula depois da condição em uma instrução `if` ou `if...else` resulta em um erro de lógica em instruções `if` de uma única seleção e um erro de sintaxe em instruções `if...else` de seleção dupla (quando a parte `if` contém uma instrução completa).

4.7 Instrução de repetição `while`

- **Instrução `while`**
  - **Repete uma ação enquanto alguma condição (para continuação do laço) permanece verdadeira**
  - **Utiliza o símbolo de agregação no diagrama de atividades UML**
    - **Une dois ou mais fluxos de atividade**
    - **Representado por um diamante (semelhante aos símbolos de decisão) mas possui:**
      - **Múltiplas setas de transição ‘incoming’,**
      - **Somente um seta de transição ‘outgoing’ e**
      - **Nenhuma das setas de transição associadas com um símbolo de agregação contém condições de guarda**

Erro comum de programação 4.3

Não fornecer, no corpo de uma instrução `while`, uma ação que consequentemente faz com que a condição na `while` se torne falsa normalmente resulta em um erro de lógica chamado *loop infinito*, no qual o *loop* nunca termina.

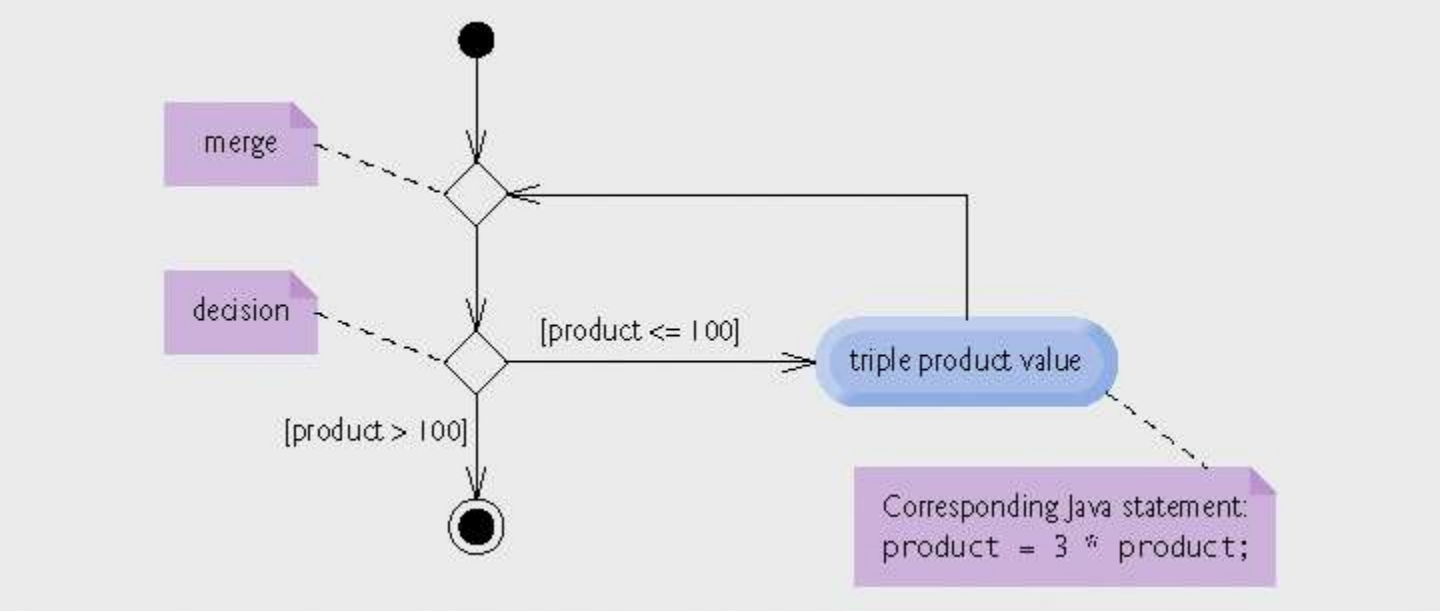
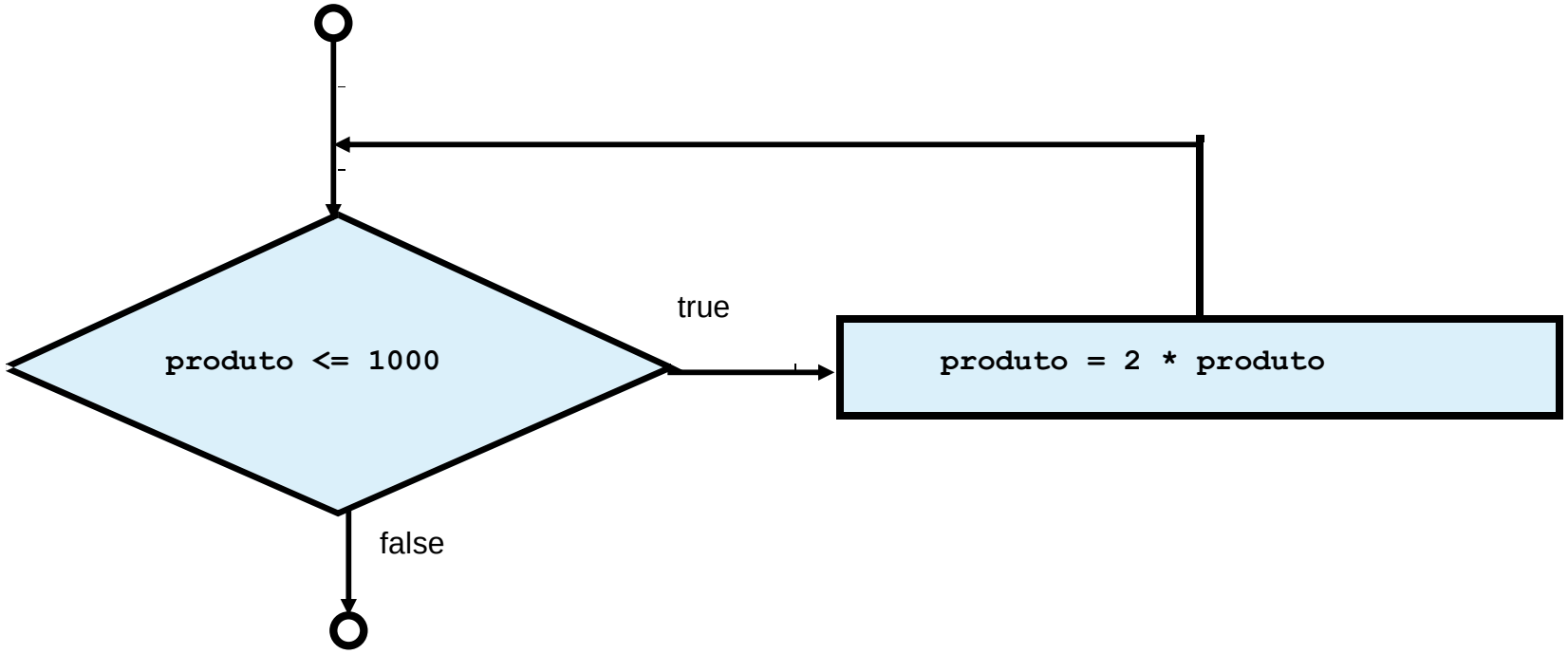


Fig. 4.4 | Diagrama de atividades UML para instrução de repetição `while` .



Fluxograma para a estrutura de repetição `while`.

4.8 Formulando algoritmos: repetição controlada por contador

- **Repetição controlada por contador**
  - **Usa uma variável contador para contar o número de vezes que um laço (loop) é iterado**
- **Divisão entre inteiros**
  - **A parte fracionária de um cálculo de divisão entre inteiros é truncada**

```
Configure o total como zero
Configure o contador de notas como um

Enquanto contador de notas for menor ou igual a dez
        Peça ao usuário para inserir a próxima nota
        Insira a próxima nota
        Adicione a nota ao total
        Adicione um ao contador de notas

Configure a média da classe como o total dividido por dez
Imprima a média da classe
```

Fig. 4.5 | Algoritmo em pseudocódigo com repetição controlada por contador para resolver o problema da média da classe.

```java
// Fig. 4.6: GradeBook.java
// Classe GradeBook que resolve o problema da média usando
// repetição controlada por contador.
import java.util.Scanner; // programa usa classe Scanner

public class GradeBook
{
   private String nomeDoCurso;// nome do curso que essa gdbook representa

   // construtor inicializa nomeDoCurso
   public GradeBook( String nome )
   {
      nomeDoCurso = nome; // inicializa nomeDoCurso
   } // fim do construtor

   // método para configurar o nome do curso
   public void setNomeDoCurso( String nome )
   {
      nomeDoCurso = nome; // armazena o nome do curso
   } // fim do método setNomeDoCurso

   // método para recuperar o nome do curso
   public String getNomeDoCurso()
   {
      return nomeDoCurso;
   } // fim do método getNomeDoCurso

```

```
28    // exibe uma mensagem de boas-vindas para o usuário GradeBook
29    public void displayMessage()
30    {
31       // getNomeDoCurso obtém o nome do curso
32       System.out.printf( "Bem-vindo a\n%s!\n\n",
33          getNomeDoCurso() );
34    } // fim do método displayMessage
35
36    // determina a média da classe com base em 10 notas inseridas pelo usuário
37    public void determineMediaDaClasse()
38    {
39       // cria Scanner para obter entrada da janela de comando
40       Scanner entrada = new Scanner( System.in );
41
42       int total; // soma das notas inseridas pelo usuário
43       int contadorNota; // número da nota a ser inserida a seguir
44       int nota; // valor da nota inserida pelo usuário
45       int media; // média das notas
46
47       // fase de inicialização
48       total = 0; // inicialize total
49       contadorNota = 1; // inicializa o contador de loops
50
```

```
51        // fase de processamento
52        while ( contadorNota <= 10 ) // executa o laço 10 vezes
53        {
54           System.out.print( "Digite a nota: " ); // prompt
55           nota = entrada.nextInt(); // insere a próxima nota
56           total = total + nota; // adiciona nota a total
57           contadorNota = contadorNota + 1; // incrementa o contador por 1
58        } // fim do while
59
60        // fase final
61        media = total / 10; // divisão por inteiros produz um resultado inteiro
62
63        // exibe o total e a média das notas
64        System.out.printf( "\nTotal de 10 notas é %d\n", total );
65        System.out.printf( "Média da classe: %d\n", media );
66     } // fim do método determineMediaDaClasse
67
68  } // fim da classe GradeBook
```

Boa prática de programação 4.5

Separe as declarações de outras instruções nos métodos com uma linha em branco para legibilidade.

Observação de engenharia de software 4.1

A experiência tem mostrado que a parte mais difícil de resolver um problema em um computador é desenvolver o algoritmo para a solução. Uma vez que o algoritmo correto foi especificado, o processo de produção de um programa Java funcional a partir do algoritmo é normalmente simples.

Erro comum de programação 4.4

Utilizar o valor de uma variável local antes de ela ser inicializada resulta em um erro de compilação. Todas as variáveis locais devem ser inicializadas antes de seus valores serem utilizados nas expressões.

Dica de prevenção de erros 4.1

Inicialize cada contador e total em sua declaração ou em uma instrução de atribuição. Normalmente, os totais são inicializados como 0. Os contadores normalmente são inicializados como 0 ou 1, dependendo de como eles são utilizados (serão mostrados exemplos de quando utilizar 0 e quando utilizar 1).

```java
// Fig. 4.7: GradeBookTest.java
// Cria o objeto da classe GradeBook e invoca seu método determineClassAverage.

public class GradeBookTest
{
   public static void main( String args[] )
   {
      // cria o objeto da classe GradeBook e
      // passa o nome do curso ao construtor
      GradeBook minhasNotas = new GradeBook(
         "Programação II" );

      minhasNotas.displayMessage(); // exibe a mensagem de boas-vindas
      minhasNotas.determineMediaDaClasse(); // calcula a média das 10 notas
   } // fim do main

} // fim da classe GradeBookTest
```

```
Bem-vindo a
Programação II!

Digite a nota: 67
Digite a nota: 78
Digite a nota: 89
Digite a nota: 67
Digite a nota: 87
Digite a nota: 98
Digite a nota: 93
Digite a nota: 85
Digite a nota: 82
Digite a nota: 100

Total de 10 notas é 846
Média da classe: 84
```

Erro comum de programação 4.5

Supor que a divisão de inteiro arredonda (em vez de truncar) pode levar a resultados incorretos. Por exemplo, 7 ÷ 4, que produz 1.75 na aritmética convencional, é truncado para 1 na aritmética de inteiros, em vez de arredondado para 2.

4.9 Formulando algoritmos: repetição controlada por sentinela

- **Repetição controlada por sentinela**
  - **Também conhecida como repetição indefinida**
  - **Utiliza um valor de sentinela (também chamado valor de sinal, valor fictício ou valor de flag)**
    - **Um valor de sentinela não pode ser um valor aceitável de entrada**

4.9 Formulando algoritmos: repetição controlada por sentinela (Cont.)

- **Refinamento passo a passo de cima para baixo (top-down)**
  - **Passo do topo: única instrução que fornece a função geral do programa**
  - **Primeiro refinamento: múltiplas instruções usando somente uma estrutura de sequência**
  - **Segundo refinamento: define variáveis específicas, utiliza estruturas de controle específicas**

Erro comum de programação 4.6

Escolher um valor de sentinela que também seja um valor legítimo de dados é um erro de lógica.

Observação de engenharia de software 4.2

Cada refinamento, bem como a própria parte superior, é uma especificação completa do algoritmo—somente o nível de detalhe varia.

Observação de engenharia de software 4.3

Muitos programas podem ser divididos logicamente em três fases: uma fase de inicialização das variáveis; uma fase de processamento que insere os valores dos dados e ajusta as variáveis do programa (por exemplo, contadores e totais) de maneira correspondente; e uma fase de término que calcula e gera a saída dos resultados finais.

Dica de prevenção de erro 4.2

Ao realizar a divisão de uma expressão cujo valor poderia ser zero, teste essa possibilidade explicitamente e trate-a adequadamente no seu programa (por exemplo, imprimindo uma mensagem de erro) em vez de permitir que o erro ocorra.

```
Inicialize total como zero
Inicialize o contador como zero

Peça ao usuário digite a primeira nota
Insira a primeira nota (possivelmente a sentinela)

Enquanto (while) o usuário não inserir a sentinela
        Adicione essa nota à soma total
        Adicione um ao contador de notas
        Peça ao usuário para inserir a próxima nota
        Insira a próxima nota (possivelmente a sentinela)

Se (if) o contador não for igual a zero
        Configure a média como o total dividido pelo contador
        Imprima a média
caso contrário (else)
        Imprima "Nenhuma nota foi inserida"
```

Fig. 4.8 | Algoritmo em pseudocódigo do problema de média da classe com repetição controlada por sentinela (flag, sinal ou fictício).

Observação de engenharia de software 4.4

Termine o processo de refinamento passo a passo de cima para baixo depois de ter especificado o algoritmo em pseudocódigo com detalhes suficientes para que você possa converter o pseudocódigo em Java. Normalmente, implementar então o programa em Java é simples e direto.

Observação de engenharia de software 4.5

Alguns programadores experientes escrevem programas sem jamais utilizar ferramentas de desenvolvimento de programa como pseudocódigo. Eles acreditam que seu objetivo final é resolver o problema em um computador e que escrever pseudocódigo só retarda a produção das saídas finais. Embora esse método talvez funcione para problemas simples e conhecidos, ele pode levar a erros sérios e atrasos em projetos grandes e complexos.

```java
// Fig. 4.9: GradeBook.java
// Classe GradeBook que resolve o problema da média da classe usando
// repetição controlada por sentinela.
import java.util.Scanner; // programa usa classe Scanner

public class GradeBook
{
   private String nomeDoCurso; // nome do curso que essa GradeBook representa

   // construtor inicializa nomeDoCurso
   public GradeBook( String nome )
   {
      nomeDoCurso = nome; // inicializa nomeDoCurso
   } // fim do construtor

   // method to set the course name
   public void setNomeDoCurso( String nome )
   {
      nomeDoCurso = nome; // armazena o nome do curso
   } // fim do método setNomeDoCurso

   // método para recuperar o nome do curso
   public String getNomeDoCurso()
   {
      return nomeDoCurso;
   } // fim do método getNomeDoCurso

```

```
28    // exibe uma mensagem de boas-vindas ao usuário
29    public void displayMessage()
30    {
31       // getNomeDoCurso obtém o nome do curso
32       System.out.printf( "Bem-vindo a\n%s!\n\n",
33          getNomeDoCurso() );
34    } // fim do método displayMessage
35
36    // determina a média de um número arbitrário de notas
37    public void determineMediaDaClasse()
38    {
39       // cria um Scanner para entrada de dados
40       Scanner entrada = new Scanner( System.in );
41
42       int total; // soma das notas
43       int contadorNotas; // número de notas inseridas
44       int nota; // valor da nota
45       double media; // número com ponto de fração decimal para a média
46
47       // fase de inicialização
48       total = 0; // inicializa o total
49       contadorNotas = 0; // inicializa o contador de loops
50
51       // fase de processamento
52       // solicita entrada e lê a nota do usuário
53       System.out.print( "Digite a nota ou -1 para sair: " );
54       nota = entrada.nextInt();
55
```

```
56        // faz um loop até ler o valor de sentinela inserido pelo usuário
57        while ( nota != -1 )
58        {
59           total = total + nota; // adiciona nota a total
60           contadorNotas = contadorNotas + 1; // incrementa contador
61
62           // solicita entrada e lê a próxima nota do usuário
63           System.out.print( "Digite a nota ou -1 para sair: " );
64           nota = entrada.nextInt();
65        } // fim do while
66
67        // fase de finalização
68        // se o usuário inseriu pelo menos uma nota...
69        if ( contadorNotas != 0 )
70        {
71           // calcula a média de todas as notas inseridas
72           media = (double) total / contadorNotas;
73
74           // exibe o total e a média (com dois dígitos de precisão)
75           System.out.printf( "\nTotal de %d notas inseridas é %d\n",
76              contadorNotas, total );
77           System.out.printf( "Média da classe: %.2f\n", media );
78        } // fim do if
79        else // caso nenhuma nota tenha sido inserida
80           System.out.println( "Nenhuma nota foi inserida" );
81     } // fim do método determineMediaDaClasse
82
83  } // fim da classe GradeBook
```

Boa prática de programação 4.6

Em um laço (loop) controlado por sentinela, os *prompts* solicitando entrada de dados devem lembrar explicitamente o usuário do valor da sentinela.

Erro comum de programação 4.7

Omitir as chaves que delimitam um bloco pode levar a erros de lógica, como loops infinitos. Para evitar esse problema, alguns programadores incluem o corpo de cada instrução de controle dentro de chaves de mesmo se o corpo contiver somente uma única instrução.

4.9 Formulando algoritmos: repetição controlada por sentinela (Cont.)

- **Operador unário de coerção (ou cast)**
  - **Cria uma cópia temporária do seu operando com um tipo de dados diferente**
    - **Exemplo: `(double)` criará uma cópia temporária de ponto-flutuante do seu operando**
  - **Conversão explícita**
- **Promoção**
  - **Conversão de um valor (p.ex. `int`) para outro tipo de dado (p.ex. `double`) para realizar um cálculo**
  - **Conversão implícita**

Erro comum de programação 4.8

O operador de coerção pode ser utilizado para converter entre tipos numéricos e primitivos, como int e double e entre tipos por referência relacionados. Aplicar uma coerção ao tipo errado pode causar erros de compilação ou erros em tempo de execução.

```java
// Fig. 4.10: GradeBookTest.java
// Cria o objeto da classe GradeBook e invoca seu método determineMediaDaClasse.

public class GradeBookTest
{
   public static void main( String args[] )
   {
      // cria o objeto da classe e
      // passa o nome do curso ao construtor
      GradeBook minhasNotas = new GradeBook(
         "Computação Gráfica" );

      minhasNotas.displayMessage(); // exibe a mensagem de boas-vindas
      minhasNotas.determineMediaDaClasse(); // encontra a média das notas
   } // fim do main

} // fim da classe GradeBookTest
```

```
Bem-vindo a
Computação Gráfica!

Digite a nota ou -1 para sair: 97
Digite a nota ou -1 para sair: 88
Digite a nota ou -1 para sair: 72
Digite a nota ou -1 para sair: -1

Total de 3 notas inseridas é 257
Média da classe: 85.67
```

4.10 Formulando algoritmos: instruções de controle aninhadas

- **Instruções de controle podem ser aninhadas (conectadas) umas as outras**
  - **Coloque uma instrução de controle no corpo de outra**

```
Inicialize as aprovações como zero
Inicialize as reprovações como zero
Inicialize o contador de alunos como um

Enquanto (while) o contador de alunos for menor ou igual a 10
   Peça ao usuário para inserir o próximo resultado do exame
   Insira o próximo resultado do exame

   Se (if) o aluno foi aprovado
      Adicione um a aprovações
   caso contrário (else)
      Adicione um a reprovações

   Adicione um ao contador de alunos

Imprima o número de aprovações
Imprima o número de reprovações

Se (if) mais de oito alunos forem aprovados
   Imprima "Elevar a taxa de matrícula"
```

Fig. 4.11 | Pseudocódigo para o problema dos resultados do exame.

```java
// Fig. 4.12: Analysis.java
// Análise dos resultados dos exames.
import java.util.Scanner; // classe usa a classe Scanner

public class Analysis
{
   public void processaResultadosDosExames()
   {
      // cria um Scanner para entrada de dados
      Scanner entrada = new Scanner( System.in );

      // inicializando variáveis nas declarações
      int aprovacoes = 0; // número de aprovações
      int reprovacoes = 0; // número de reprovações
      int contadorAlunos = 1; // contador de alunos
      int resultado; // um resultado do exame (obtém valor a partir do usuário)

      // processa 10 alunos usando loop controlado por contador
      while ( contadorAlunos <= 10 )
      {
         // solicita ao usuário entrada e obtém seu valor
         System.out.print( "Digite o resultado (1 = passou, 2 = reprovou): " );
         resultado = entrada.nextInt();

```

```
25          // if...else aninhado em um while
26          if ( resultado == 1 )           // if resultado 1,
27             aprovacoes = aprovacoes + 1;     // incrementa aprovacoes;
28          else                            // caso resultado não seja 1, então
29             reprovacoes = reprovacoes + 1; // incrementa reprovacoes
30
31          // incrementa contadorAlunos até o loop terminar
32          contadorAlunos = contadorAlunos + 1;
33       } // fim do while
34
35       // fase de término; prepara e exibe os resultados
36       System.out.printf( "Aprovados: %d\nReprovados: %d\n", aprovacoes, reprovacoes );
37
38       // determina se mais de 8 alunos passaram
39       if ( aprovacoes > 8 )
40          System.out.println( "Aumentar taxa de matrícula" );
41    } // fim do método processaResultadosDosExames
42
43 } // fim da classe Analysis
```

Dica de prevenção de erro 4.3

Inicializar variáveis locais quando são declaradas ajuda o programador a evitar quaisquer erros de compilação que poderiam surgir de tentativas para utilizar dados não-inicializados. Embora o Java não exija que as inicializações das variáveis locais sejam incorporadas a declarações, ele exige que variáveis locais sejam inicializadas antes de seus valores serem utilizados em uma expressão.

```java
// Fig. 4.13: AnalysisTest.java
// Programa de teste para a classe Analysis.

public class AnalysisTest
{
   public static void main( String args[] )
   {
      Analysis teste = new Analysis(); // cria o objeto Analysis
      teste.processaResultadosDosExames(); // chamada de método
   } // fim do main

} // fim da classe AnalysisTest
```

```
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 2
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Aprovados: 9
Reprovados: 1
Aumentar taxa de matrícula
```

Mais de 8 alunos passaram na prova

```
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 2
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 2
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 2
Digite o resultado (1 = passou, 2 = reprovou): 2
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Digite o resultado (1 = passou, 2 = reprovou): 1
Aprovados: 6
Reprovados: 4
```

4.11 Operadores de atribuição compostos

- **Operadores de atribuição compostos**
  - **Uma instrução de atribuição na forma:**
    ***variável* = *variável operador expressão* ;**
    onde *operador* é `+`, `-`, `*`, `/` ou `%` pode ser escrita como:
    ***variável operador*= *expressão* ;**
  - **Exemplo: `c = c + 3;` pode ser escrita como `c += 3;`**
    - **Essa instrução adiciona `3` ao valor na variável `c` e armazena o resultado na variável `c`**

Operador de atribuição	Expressão de exemplo	Explicação	Atribuições
Suponha: `int c = 3, d = 5, e = 4, f = 6, g = 12;`			
`+=`	`c += 7`	`c = c + 7`	10 a c
`-=`	`d -= 4`	`d = d - 4`	1 a d
`*=`	`e *= 5`	`e = e * 5`	20 a e
`/=`	`f /= 3`	`f = f / 3`	2 a f
`%=`	`g %= 9`	`g = g % 9`	3 a g

Fig. 4.14 | Operadores de atribuição compostos aritméticos.

4.12 Operadores de incremento e decremento

- **Operadores de incremento e decremento unários**
  - **Operador de incremento unário (++) adiciona um ao seu operando**
  - **Operador de decremento unário (−−) subtrai um do seu operando**
  - **Operador de pré-incremento (e pré-decremento)**
    - **Altera o valor do seu operando, então utiliza o novo valor do operando na expressão a qual o operando aparece**
  - **Operador de pós-incremento (e pós-decremento)**
    - **Utiliza o valor atual do seu operando na expressão a qual o operando aparece, então altera o valor do operando**

Boa prática de programação 4.7

Diferente dos operadores binários, os operadores de incremento e decremento unários devem ser colocados ao lado dos seus operandos, sem espaços no meio.

Operador	Chamado	Expressão de exemplo	Explicação
`++`	Pré-Incremento	`++a`	Incrementa `a` por `1`, então usa o novo valor de `a` na expressão em que `a` reside.
`++`	Pós-Incremento	`a++`	Usa o valor atual de `a` na expressão em que `a` reside, então incrementa `a` por `1`.
`--`	Pré-Decremento	`--b`	Decrementa `b` por `1`, então usa o novo valor de `b` na expressão em que `b` reside.
`--`	Pós-Decremento	`b--`	Usa o valor atual de `b` na expressão em que `b` reside, então decrementa `b` por `1`.

Fig. 4.15 | Operadores de incremento e decremento.

```java
// Fig. 4.16: Increment.java
// Operadores e pré-incremento e pós-incremento.

public class Increment
{
   public static void main( String args[] )
   {
      int c;

      // demonstra o operador de pós-incremento
      c = 5; // atribui 5 a variável c
      System.out.println( c );   // imprime 5
      System.out.println( c++ ); // imprime 5 então pós-incrementa
      System.out.println( c );   // imprime 6

      System.out.println(); // pula uma linha

      // demonstra o operador de pré-incremento
      c = 5; // atribui 5 a variável c
      System.out.println( c );   // imprime 5
      System.out.println( ++c ); // pré-incrementa e então imprime 6
      System.out.println( c );   // imprime 6

   } // fim do main

} // fim dad classe Increment
```

```
5
5
6

5
6
6
```

Erro comum de programação 4.9

Tentar utilizar o operador de incremento ou de decremento em uma expressão diferente daquela a que um valor pode ser atribuído é um erro de sintaxe. Por exemplo, escrever `++(x + 1)` é um erro de sintaxe porque `(x + 1)` não é uma variável.

Operadores						Associatividade	Tipo
`++`	`--`					Da direita para a esquerda	Unário pós-fixo
`++`	`--`	`+`	`-`	`( tipo )`		Da direita para a esquerda	Unário pré-fixo
`*`	`/`	`%`				Da esquerda para a direita	Multiplicativo
`+`	`-`					Da esquerda para a direita	Aditivo
`<`	`<=`	`>`	`>=`			Da esquerda para a direita	Relacional
`==`	`!=`					Da esquerda para a direita	Igualdade
`?:`						Da direita para a esquerda	Ternário condicional
`=`	`+=`	`-=`	`*=`	`/=`	`%=`	Da direita para a esquerda	Atribuição

Fig. 4.17 **Precedência e associatividade dos operadores discutidos até agora.**

4.13 Tipos primitivos

- **Java é uma linguagem fortemente tipificada**
  - **Todas as variáveis têm de ter um tipo**
- **Tipos primitivos em Java são portáveis através de todas as plataformas que suportam Java**

Dica de portabilidade 4.1

Diferente de C e C++, os tipos primitivos em Java são portáveis enter todas as plataformas de computador que suportam Java. Graças a isso e a muitos outros recursos da portabilidade do Java, um programador pode escrever um programa uma vez e estar certo de que ele executará em qualquer plataforma de computador que suporte o Java. Essa capacidade é referida às vezes como *WORA* (*Write Once, Run Anywhere* – Escreva uma vez, execute em qualquer lugar).

Anexo: Palavras reservadas em Java

Java Keywords				
abstract	**boolean**	**break**	**byte**	**case**
catch	**char**	**class**	**continue**	**default**
do	**double**	**else**	**extends**	**false**
final	**finally**	**float**	**for**	**if**
implements	**import**	**instanceof**	**int**	**interface**
long	**native**	**new**	**null**	**package**
private	**protected**	**public**	**return**	**short**
static	**super**	**switch**	**synchronized**	**this**
throw	**throws**	**transient**	**true**	**try**
void	**volatile**	**while**		
Palavras reservadas, mas não usadas, por Java				
const	**goto**			
Java keywords.				

Referências

Adaptado de: “Teaching Resources for Java How to Program, 6/e”.

Deitel & Deitel. Java como programar, 6 ed.